

SWC-104: Unchecked Call Return Value

Exploit Report

Samuel Lechner
12007552

Philipp Gorke
12022511

21 December 2023

1 Introduction

1.1 Weakness vs. Vulnerability

A **weakness** of a smart contract is a piece of code, that potentially could be used to exploit. If a weakness actually lead to an exploit, it should be considered as **vulnerability**. That means: Every vulnerability is also a weakness but NOT vice versa.

1.2 Precautions

The LVA Team provided a couple of contracts that had to be examined. All contracts can be found in this repository. Our branch was: swc-104. First, it was from high priority to get an overall overview so the contracts were inspected manually to get a first feeling for their structure.

1.3 Preprocessing

Subsequently, a tool named **Mythril** was downloaded. This tool was recommended by the LVA team to scan the contracts for unchecked call return values. There have been some technical difficulties with Docker but after some time it worked properly. This tool was from great help, in particular, for inspecting larger contracts with hundreds, sometimes thousands of lines. After that we looked at the linked papers from the other tools and found out they essentially scanned the contracts the same way. What they did was: Find any low-level calls and check whether there is a check for it afterwards.

1.4 Setup

The linked repository above was imported in an IDE (Jetbrains' IntelliJ) which went as easy as expected. The following workflow was then applied on every contract:

1. Assign contract to group member
2. Examine contract manually (optionally also by tool)
3. Is an unchecked call return value weakness found?
 - If yes: Is weakness also a vulnerability? To find this out: Every low-level call had to be checked for logic and state changes afterwards. For interesting files, which were hard to verify, we took the code to remix and tested a lot with exploit contracts.
 - If yes: Copy `.sol` file and mark the critical lines in the source code
 - If no: continue
 - If no: continue
4. Create `.txt` file to log the status and provide further explanation.
5. Update status of this contract in the `-svc` file.

2 Exploit Creation

2.1 Extra Short Recap of Unchecked Call Return Value Definition

A smart contract (caller) can call other addresses (callee). There are various reasons: Some calls aim to invoke methods in the callee while others only want to transfer ETH. A distinction can be made between Low- and High-Level calls. While Low level have corresponding OP-Codes that can be encoded to byte codes, High level calls are more "abstract" and therefore offer more functionality including error handling. Therefore, Low level calls are dangerous to use in a sense that the solidity programmer has to make sure that the method's return value (true on success, false on fail) has to be checked manually.

Here's a simplified example:

```

1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.20;
3 contract ExampleContract {
4     function foo() {
5         address to;
6         bool status;
7         bytes memory input;
8         bytes memory output;
9         // the next line by itself is considered as unchecked
10        (status, output) = to.staticcall(input);
11        // to avoid this, simply add:
12        require(status);
13        // everything below will only be executed if status is true
14        // else, the transaction will be reverted
15    }
16 }
17
```

2.2 Exploit ideas

Vulnerable contracts don't check for the call's outcome, but "assume" the expected result and continue execution. An attacker could purposely make low-level calls fail, for example by providing an exact amount of gas or make their contract revert the call. Without a revert, executed operations will be mined on the blockchain and will change the contract's state permanently, even though the low-level call failed.

There are also cases where the call outcome does not matter (i.e. the contract's state does not depend on the call's return value) - however, these cases will be discussed later.

3 Results

3.1 Vulnerable Contracts

- AnkleAcceptHead.sol
- SecurityTest.sol

3.2 Non-Exploitable Contracts

- AuthenticatedProxy.sol
- BatchCaller.sol
- BombDisposal.sol
- CoreVault_2.sol
- CoreVault.sol
- CorexVault.sol
- decoreVault_2.sol
- decoreVault.sol
- Demons.sol
- dfcoreVault.sol
- EncoreVault_2.sol
- EncoreVault.sol
- ethpay.sol
- FeeApprover.sol
- FlashBotLowGas.sol

- Forwarder.sol
- fuelVault.sol
- HdcoreVault.sol
- KernelVault.sol
- LilPix.sol
- MeterMaid.sol
- MultiSend.sol
- QoreVault.sol
- SaltAndSatoshiBondingCurve.sol
- SmartVestingDigg.sol
- testSend_2.sol
- testSend.sol
- TMELocker.sol
- Toolkit.sol
- transferThroughContract.sol
- UnicStakingV3.sol
- upcoreVault.sol
- upcoreVaultV2.sol
- Vether_2.sol
- Vether_3.sol
- Vether3.sol
- Vether4_2.sol
- Vether_4.sol
- Vether4.sol
- Vether_5.sol
- Vether_6.sol
- Vether.sol
- XCOREVault_2.sol
- XCOREVault_3.sol
- XCOREVault.sol

3.3 Undecidable Contracts

- FlashFloss.sol We were not sure where to classify this contract. The contract's purpose is to take the ether and send it back and remember who the highest payer was so far. However, when a call fails, the ether is not returned and another user can take the ether instead. We were not sure if that's a vulnerability or not, especially since this contract is obviously not a serious one.

3.4 Optional Fixes

Unfortunately, both contracts were not vulnerable because of swc-104, but other weaknesses.

- AnkleAcceptHead.sol: As stated in the `.txt` file in the repository, the exploit could be fixed by adding modifier in the `vague()` method. Although this contract can be exploited, one should note that it is not due to an unchecked call. The problem in this contract is, that everyone can call the `vague()` method and purge the contract's balance.
- SecurityTest.sol: Reentrancy weaknesses could simply be fixed by adding a "Reentrancy guard" as a modifier like this website suggests.

4 Discussion

4.1 Conclusions

Statistically speaking, only 0 out of 48 contracts could be classified as exploitable due to unchecked call return values, but 2 due to other security issues, which is approximately 4%. A single contract proved to be undecidable, resulting in its classification as "unknown". The amount of contracts that are affected by the weakness is higher, however, it is important to note that in this case the contracts could not be exploited (remember: weakness $\neq$ vulnerability). The low rate however, does not verify that the weakness is uncommon and can be overlooked. The tools often return contracts that are trivially secure and do not contain any weakness. Proxy contracts and contract with libraries are also marked as "maybe vulnerable" by the tools. The vulnerability is very specific and an exact sequence of logic has to occur to find actual vulnerable contracts.

4.2 Lessons Learned

The tool `Mythril` that was used for automatically checking the contracts for the given vulnerability turned out to be a great assistance. However, we quickly realized that it finds every weaknesses, but in fact, we were rather looking for a true vulnerability. In other words, the tool helped finding the interesting pieces of code but couldn't decide whether an unchecked call is dangerous or not in each specific case.

Furthermore, many of the given contracts include common libraries like **Address**, **Console** (...) that are registered as vulnerable by the tool. Although there indeed exists unchecked calls, they cannot be used for exploits. They had to be checked anyways on each contract. In our opinions, it would be quite useful if these common libraries were whitelisted in **Mythril**.

Nevertheless, it's definitely good practice to verify each low-level call. Not only in solidity, but in every system, validating input should be a requirement.

4.3 Open Challenges

Finally it is to say, there could only be found very few weaknesses resulting from unchecked calls. Nevertheless, solidity programmers should maintain vigilance in verifying return values diligently. An immensely beneficial support would involve IDEs such as IntelliJ conducting static analyses of solidity code, thereby preemptively warning programmers during the development phase. In addition, tools are still very prone to false-negatives, such as the usage of common libraries or proxy contracts. If those tools would be more precise and filter those contracts beforehand, the rate could definitely be improved drastically.